

MANUAL HYPERPARAMETER SELECTION FORM

2D CNN Model — Dogs vs. Cats Image Classification

Name: Federico M. Griño

Program: DIT

Use this form to record, justify, and approve each hyperparameter value chosen by hand before a training run. The "Notebook Default" column shows the value currently used in `Loading_Dataset_and_CNN.ipynb` for reference — enter the value actually used for this run under "Chosen Value", even if it matches the default.

1. Project / Experiment Information

Project / Experiment Title	2D CNN Model - Dogs vs. Cats Image Classification
Model Architecture	2D Convolutional Neural Network (Conv2D)
Dataset Used	Kaggle Dogs vs. Cats images (Dog/ and Cat/ folders)
Notebook / File Reference	Loading_Dataset_and_CNN.ipynb
Prepared By	Federico M. Griño
Date	September 17, 2026
Experiment / Trial No.	3 trials

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	e.g. 50 / 80 / 100 / 128 / 150	100	Trial 1: 50 Trial 2: 80 Trial 3: 100	
Color Mode	Grayscale / RGB	Grayscale	Trial 1-3: Grayscale	
Pixel Normalization	X/255.0 / none	X/255.0	X/255.0	
Shuffle Training Data	Yes / No	Yes (random.shuffle)	Yes (random.shuffle)	
Held-out Test Set (separate from training)	Yes / No	No (validation_split only)	Yes	
Validation Split (of training set)	0.0 - 0.3	0.2	0.2	

3. Model Architecture Hyperparameters

Trial 1

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	16 / 32 / 64 / 128	14			Trial 1: 0.6510 Trial 2: 0.7343 Trial 3: 0.5704	Trial 1: 0.6302 Trial 2: 0.7165 Trial 3: 0.5500
Conv2D Layer 1 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)			
Conv2D Layer 1 – Activation	relu / tanh / elu / leaky_relu	relu	relu			
Pooling Layer 1 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)			
Dense (Fully Connected) Units	16 / 32 / 64 / 128	64	64			
Dropout Rate	0.0 - 0.5	None (not used)	None (0.0)			
Output Layer Activation	sigmoid (binary) / softmax (multi-class)	sigmoid	sigmoid			

4. Training Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	binary_crossentropy / categorical_crossentropy	binary_crossentropy	binary_crossentropy		Trial 1: 0.6510 Trial 2: 0.7343 Trial 3: 0.5704	Trial 1: 0.6302 Trial 2: 0.7165 Trial 3: 0.5500
Optimizer	adam / sgd / rmsprop	adam	adam			
Learning Rate	0.1 / 0.01 / 0.001 / 0.0001	0.001 (Adam default)	0.001 (Adam default)			
Batch Size	8 / 16 / 32 / 64 / 128	32	128			
Number of Epochs	e.g. 10 - 200	200	10			
Early Stopping	Yes (patience = ____) / No	No	No			

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy

5. Evaluation Results (after training with chosen values) - Final Model

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy	Trial 1: 0.6302, Trial 2: 0.7343, Trial 3: 0.5704	
Validation Accuracy	Trial 1: 0.6302, Trial 2: 0.7165, Trial 3: 0.5500	
Test Accuracy	Trial 1: 0.6218, Trial 2: 0.7063, Trial 3: 0.5600	
Test Loss	Trial 1: 0.6708, Trial 2: 0.5782, Trial 3: 0.6857	
Precision / Recall / F1 (macro avg)	Trial 1: 0.6733, 0.6218, 0.5914 Trial 2: 0.7254, 0.7094, 0.7041 Trial 3: 0.5784, 0.5600, 0.5326	

5. a. Paste here all screenshots/figures of the tuned model

Training 1: Localhost Run, Image Size 50, Epoch 10

```
[8]: DATADIR = "/mnt/d/DIT/Bioinformatics/Code/PetImages/PetImages" # <- change this to the path that contains your Dog/ and Cat/ folders
# this is the directory of your datasets
CATEGORIES = ["Dog", "Cat"] # Defining the label as "Dog" or "cat"

for category in CATEGORIES:
    path = os.path.join(DATADIR, category) # path to cats or dogs dir
    for img in os.listdir(path):
        img_array = cv2.imread(os.path.join(path, img), cv2.IMREAD_GRAYSCALE)
        if img_array is None:
            # a handful of files in the Kaggle dataset are corrupted/unreadable - skip them
            continue

        # reading the image in the path and transforming it into an array
        # cv2.IMREAD_GRAYSCALE converts the image to grayscale so the CNN only has
        # to learn from one color channel instead of three (faster, smaller model)

        plt.imshow(img_array, cmap="gray")
        plt.show()
        break
    break
```

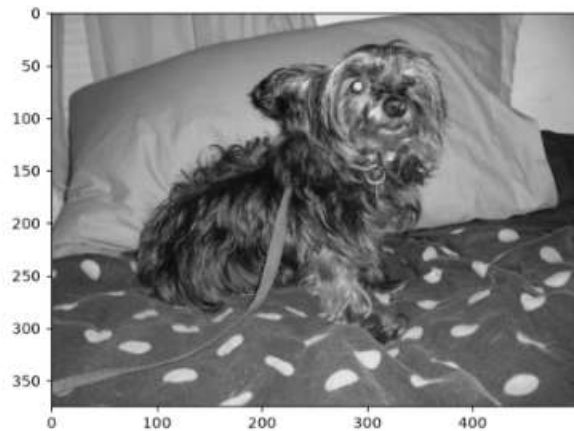


Image Inspection

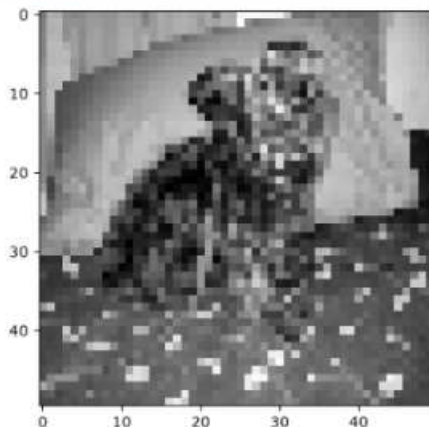
```
[9]: print(img_array) # Displaying the image in array form (just for checking)
print(img_array.shape) # checking the resolution of the image

[[117 117 119 ... 133 132 132]
 [118 117 119 ... 135 134 134]
 [119 118 120 ... 137 136 136]
 ...
 [ 79  74  73 ...  80  76  73]
 [ 78  72  69 ...  72  73  74]
 [ 74  71  70 ...  75  73  71]]
(375, 500)
```

Image Resizing

```
[10]: IMG_SIZE = 50 # initializing variable "IMG_SIZE" as 100

new_array = cv2.resize(img_array, (IMG_SIZE, IMG_SIZE))
plt.imshow(new_array, cmap='gray')
plt.show()
```



Assigning Training Data

```
[11]: training_data = [] # assigning training data

def create_training_data():
    for category in CATEGORIES:
        path = os.path.join(DATADIR, category) # path to cats or dogs dir
        class_num = CATEGORIES.index(category)
        for img in os.listdir(path):
            try:
                img_array = cv2.imread(os.path.join(path, img), cv2.IMREAD_GRAYSCALE)
                new_array = cv2.resize(img_array, (IMG_SIZE, IMG_SIZE))
                training_data.append([new_array, class_num])
            except Exception as e:
                pass

create_training_data()
```

```
Corrupt JPEG data: 399 extraneous bytes before marker 0xd9
Corrupt JPEG data: 226 extraneous bytes before marker 0xd9
Corrupt JPEG data: 162 extraneous bytes before marker 0xd9
Warning: unknown JFIF revision number 0.00
Corrupt JPEG data: 1403 extraneous bytes before marker 0xd9
Corrupt JPEG data: 254 extraneous bytes before marker 0xd9
Corrupt JPEG data: 2230 extraneous bytes before marker 0xd9
Corrupt JPEG data: 65 extraneous bytes before marker 0xd9
Corrupt JPEG data: 239 extraneous bytes before marker 0xd9
Corrupt JPEG data: 214 extraneous bytes before marker 0xd9
Corrupt JPEG data: 128 extraneous bytes before marker 0xd9
Corrupt JPEG data: 99 extraneous bytes before marker 0xd9
Corrupt JPEG data: 1153 extraneous bytes before marker 0xd9
```

Checking the Number of Training Data

```
[12]: print(len(training_data))

24997
```

Checking the Number of Training Data

```
[12]: print(len(training_data))

24997
```

Randomizing/Shuffling the Data

```
[13]: import random
      random.shuffle(training_data)

      for sample in training_data[:10]:
          print(sample[1])

0
0
1
0
1
0
0
0
0
0
```

• • • • •

Using Pickle (Loading It Back)

```
[10]: pickle_in = open('a.pickle', 'rb')
      # = pickle.load(pickle_in)

      K[1]

[11]: array([[109],
           [112],
           [119],
           ...,
           [ 47],
           [ 13],
           [205]],

          [[117],
           [116],
           [116],
           ...,
           [127],
           [ 49],
           [137]],

          [[115],
           [118],
           [119],
           ...,
           [136],
           [138],
           [131]],

          ...,

          [[149],
           [150],
           [153],
           ...,
           [148],
           [148],
           [136]],

          [[144],
           [142],
           [146],
           ...,
           [133],
           [134],
           [139]],

          [[140],
           [145],
           [147],
           ...,
           [137],
           [139],
           [134]]], dtype=uint8, shape=(48, 48, 1), #type=uint8)
```

```
[10]: model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 48, 48, 2)	20
activation (Activation)	(None, 48, 48, 2)	0
max_pooling2d (MaxPooling2D)	(None, 24, 24, 2)	0
conv2d_1 (Conv2D)	(None, 22, 22, 64)	1,216
activation_1 (Activation)	(None, 22, 22, 64)	0
max_pooling2d_1 (MaxPooling2D)	(None, 11, 11, 64)	0
conv2d_2 (Conv2D)	(None, 9, 9, 64)	36,928
activation_2 (Activation)	(None, 9, 9, 64)	0
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 64)	0
flatten (Flatten)	(None, 1024)	0
dense (Dense)	(None, 64)	65,600
dense_1 (Dense)	(None, 64)	4,160
dense_2 (Dense)	(None, 1)	65
activation_3 (Activation)	(None, 1)	0

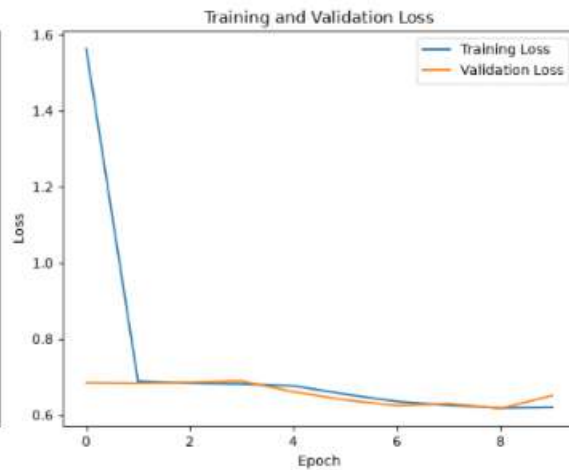
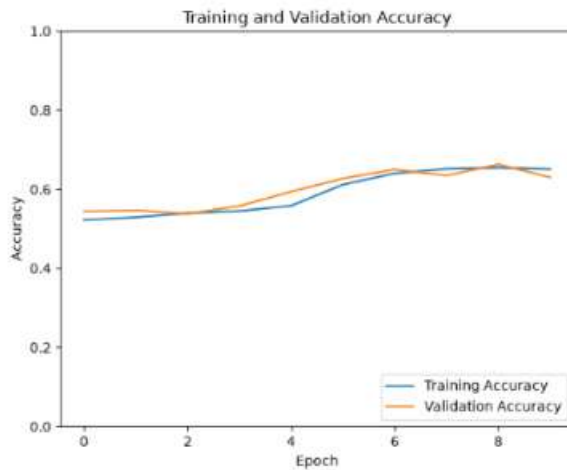
Total params: 107,989 (421.83 KB)

Trainable params: 107,989 (421.83 KB)

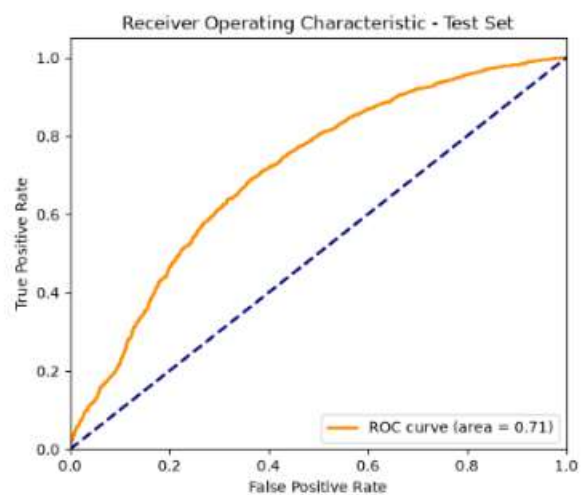
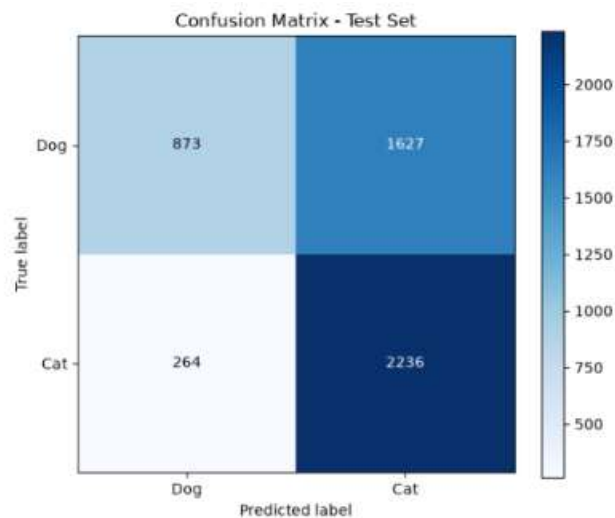
Non-trainable params: 0 (0.00 B)

```
[21]: # Train the CNN using only the training portion.
# validation_split=0.20 creates a validation set from X_train/y_train.
history = model.fit(
    X_train,
    y_train,
    batch_size=128,
    epochs=10,
    validation_split=0.2,
    shuffle=True,
    verbose=1
)
```

```
Epoch 1/10
125/125 — 10s 57ms/step - accuracy: 0.5220 - loss: 1.5622 - val_accuracy: 0.5437 - val_loss: 0.6845
Epoch 2/10
125/125 — 7s 60ms/step - accuracy: 0.5292 - loss: 0.6802 - val_accuracy: 0.5465 - val_loss: 0.6833
Epoch 3/10
125/125 — 9s 72ms/step - accuracy: 0.5397 - loss: 0.6834 - val_accuracy: 0.5378 - val_loss: 0.6853
Epoch 4/10
125/125 — 11s 87ms/step - accuracy: 0.5449 - loss: 0.6812 - val_accuracy: 0.5577 - val_loss: 0.6889
Epoch 5/10
125/125 — 9s 75ms/step - accuracy: 0.5584 - loss: 0.6763 - val_accuracy: 0.5940 - val_loss: 0.6608
Epoch 6/10
125/125 — 10s 78ms/step - accuracy: 0.6120 - loss: 0.6543 - val_accuracy: 0.6273 - val_loss: 0.6391
Epoch 7/10
125/125 — 10s 82ms/step - accuracy: 0.6404 - loss: 0.6350 - val_accuracy: 0.6500 - val_loss: 0.6242
Epoch 8/10
125/125 — 13s 107ms/step - accuracy: 0.6517 - loss: 0.6251 - val_accuracy: 0.6348 - val_loss: 0.6296
Epoch 9/10
125/125 — 11s 88ms/step - accuracy: 0.6546 - loss: 0.6180 - val_accuracy: 0.6630 - val_loss: 0.6166
Epoch 10/10
125/125 — 10s 83ms/step - accuracy: 0.6510 - loss: 0.6196 - val_accuracy: 0.6302 - val_loss: 0.6508
```



Confusion Matrix (Test Set):
[[873 1627]
[264 2236]]



```
f"({f1 * 100:.2f}%"
)
print("=" * 65)
```

=====

FINAL CNN MODEL EVALUATION

=====

```
Training Accuracy: 0.6518 (65.18%)
Validation Accuracy: 0.6382 (63.82%)
Test Accuracy: 0.6218 (62.18%)
Test Loss: 0.6788
Precision (macro): 0.6733 (67.33%)
Recall (macro): 0.6218 (62.18%)
F1-score (macro): 0.5914 (59.14%)
=====
```

Prediction Inference

Now that the model is loaded, we can use it to make predictions on new, unseen data. For demonstration purposes, I'll take a few samples from your existing `X` data to illustrate how to get predictions from the `loaded_model`.

```
[25]: # Select a small subset of the unseen test set for inference
test_samples = X_test[:5]

# Make predictions using the loaded model
inference_predictions = loaded_model.predict(test_samples, verbose=0)

print("Predictions for unseen test samples:")
for i, prediction in enumerate(inference_predictions):
    predicted_class = 'Cat' if prediction[0] > 0.5 else 'Dog'
    probability = prediction[0] * 100
    print(f"Sample {i+1}: Predicted Class = {predicted_class} (Cat probability: {probability:.2f}%)")

# Display the images and predictions
plt.figure(figsize=(10, 4))
for i in range(min(5, len(test_samples))):
    plt.subplot(1, 5, i + 1)
    plt.imshow(test_samples[i].reshape(IMG_SIZE, IMG_SIZE), cmap='gray')
    predicted_class = 'Cat' if inference_predictions[i][0] > 0.5 else 'Dog'
    plt.title(f"Pred: {predicted_class}")
    plt.axis('off')
plt.tight_layout()
plt.show()
```

Predictions for unseen test samples:
Sample 1: Predicted Class = Cat (Cat probability: 81.86%)
Sample 2: Predicted Class = Cat (Cat probability: 65.74%)
Sample 3: Predicted Class = Cat (Cat probability: 51.17%)
Sample 4: Predicted Class = Dog (Cat probability: 49.81%)
Sample 5: Predicted Class = Cat (Cat probability: 72.54%)



[25]

Training 2: Localhost Run, Image Size 80, Epoch 10

```
[#]: DATADIR = "/mnt/d/DIT/Bioinformatics/Code/PetImages/PetImages" # <- change this to the path that contains your dog/ and cat/ folders
# this is the directory of your datasets
CATEGORIES = ["Dog", "Cat"] # Defining the Label as "Dog" or "Cat"

for category in CATEGORIES:
    path = os.path.join(DATADIR, category) # path to cats or dogs dir
    for img in os.listdir(path):
        img_array = cv2.imread(os.path.join(path, img), cv2.IMREAD_GRAYSCALE)
        if img_array is None:
            # a handful of files in the Kaggle dataset are corrupted/unreadable - skip them
            continue

        # reading the image in the path and transforming it into an array
        # cv2.IMREAD_GRAYSCALE converts the image to grayscale so the CNN only has
        # to learn from one color channel instead of three (faster, smaller model)

        plt.imshow(img_array, cmap="gray")
        plt.show()
        break
    break
```

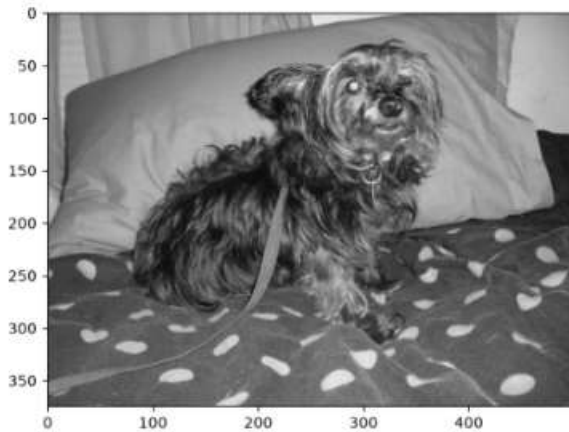


Image Inspection

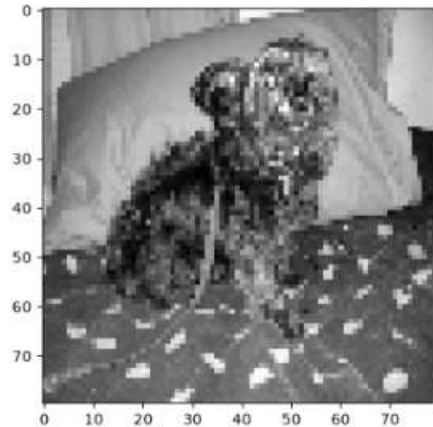
```
[9]: print(img_array)      # Displaying the image in array form (just for checking)
     print(img_array.shape) # checking the resolution of the image

[[117 117 119 ... 133 132 132]
 [118 117 119 ... 135 134 134]
 [119 118 120 ... 137 136 136]
 ...
 [ 79  74  73 ...  80  76  73]
 [ 78  72  69 ...  72  73  74]
 [ 74  71  70 ...  75  73  71]]
(375, 500)
```

Image Resizing

```
[10]: IMG_SIZE = 80 # initializing variable "IMG_SIZE" as 100

new_array = cv2.resize(img_array, (IMG_SIZE, IMG_SIZE))
plt.imshow(new_array, cmap='gray')
plt.show()
```



Assigning Training Data

```
[11]: training_data = [] # assigning training data

def create_training_data():
    for category in CATEGORIES:
        path = os.path.join(DATADIR, category) # path to cats or dogs dir
        class_num = CATEGORIES.index(category)
        for img in os.listdir(path):
            try:
                img_array = cv2.imread(os.path.join(path, img), cv2.IMREAD_GRAYSCALE)
                new_array = cv2.resize(img_array, (IMG_SIZE, IMG_SIZE))
                training_data.append((new_array, class_num))
            except Exception as e:
                pass

create_training_data()
```

```
Corrupt JPEG data: 399 extraneous bytes before marker 0xd9
Corrupt JPEG data: 226 extraneous bytes before marker 0xd9
Corrupt JPEG data: 162 extraneous bytes before marker 0xd9
Warning: unknown JFIF revision number 0.00
Corrupt JPEG data: 1483 extraneous bytes before marker 0xd9
Corrupt JPEG data: 254 extraneous bytes before marker 0xd9
Corrupt JPEG data: 2230 extraneous bytes before marker 0xd9
Corrupt JPEG data: 65 extraneous bytes before marker 0xd9
Corrupt JPEG data: 239 extraneous bytes before marker 0xd9
Corrupt JPEG data: 214 extraneous bytes before marker 0xd9
Corrupt JPEG data: 128 extraneous bytes before marker 0xd9
Corrupt JPEG data: 99 extraneous bytes before marker 0xd9
Corrupt JPEG data: 1153 extraneous bytes before marker 0xd9
```

Checking the Number of Training Data

```
[12]: print(len(training_data))

24997
```

Randomizing/Shuffling the Data

```
5]: import random
    random.shuffle(training_data)

    for sample in training_data[:10]:
        print(sample[1])
```

```
0
0
1
1
1
1
0
0
1
1
```

```
[16]: pickle_in = open('X.pickle', 'rb')
    X = pickle.load(pickle_in)

    X[1]
```

```
[16]: array([[[ 23],
               [ 9],
               [ 13],
               ...,
               [ 31],
               [ 67],
               [ 14]],

              [[ 29],
               [ 15],
               [ 15],
               ...,
               [ 27],
               [ 92],
               [ 14]],

              [[ 41],
               [ 61],
               [ 17],
               ...,
               [ 38],
               [ 99],
               [ 30]],

              ...,

              [[105],
               [121],
               [117],
               ...,
               [117],
               [ 88],
               [122]],

              [[ 65],
               [105],
               [ 82],
               ...,
               [ 42],
               [ 87],
               [101]],

              [[157],
               [103],
               [ 86],
               ...,
               [ 98],
               [ 90],
               [130]]], shape=(80, 80, 1), dtype=uint8)
```

```
[18]: # Required imports for dataset splitting and evaluation
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, roc_curve, auc

# Ensure labels are a NumPy array
y = np.asarray(y).astype(int)

# Split the dataset into training/development data and a completely unseen test set
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print(f"Total samples: {len(X):,}")
print(f"Training samples: {len(X_train):,}")
print(f"Test samples: {len(X_test):,}")
print("\nThe training data will use validation_split=0.20, so the final proportions are approximately:")
print("  Training: 64%")
print("  Validation: 16%")
print("  Test: 20%")

Total samples: 24,997
Training samples: 19,997
Test samples: 5,000
```

The training data will use validation_split=0.20, so the final proportions are approximately:

```
Training: 64%
Validation: 16%
Test: 20%
```

```
[20]: model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 78, 78, 2)	20
activation (Activation)	(None, 78, 78, 2)	0
max_pooling2d (MaxPooling2D)	(None, 39, 39, 2)	0
conv2d_1 (Conv2D)	(None, 37, 37, 64)	1,216
activation_1 (Activation)	(None, 37, 37, 64)	0
max_pooling2d_1 (MaxPooling2D)	(None, 18, 18, 64)	0
conv2d_2 (Conv2D)	(None, 16, 16, 64)	36,928
activation_2 (Activation)	(None, 16, 16, 64)	0
max_pooling2d_2 (MaxPooling2D)	(None, 8, 8, 64)	0
flatten (Flatten)	(None, 4096)	0
dense (Dense)	(None, 64)	262,208
dense_1 (Dense)	(None, 64)	4,160
dense_2 (Dense)	(None, 1)	65
activation_3 (Activation)	(None, 1)	0

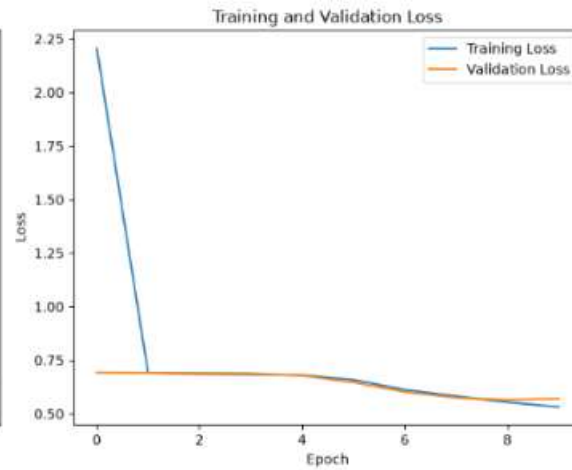
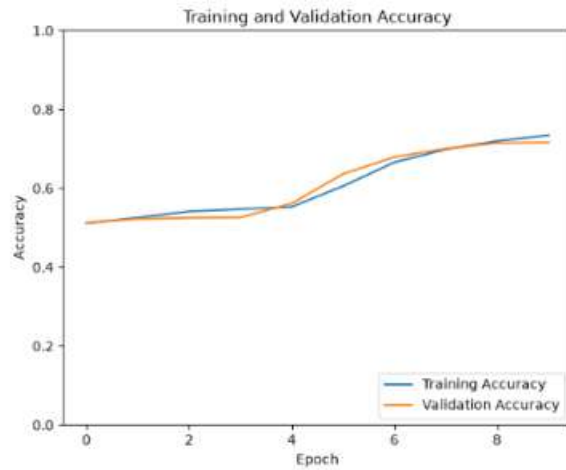
Total params: 304,597 (1.16 MB)

Trainable params: 304,597 (1.16 MB)

Non-trainable params: 0 (0.00 B)

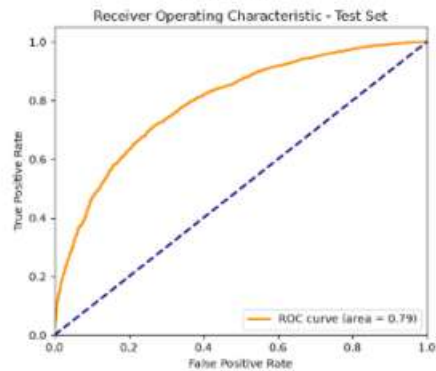
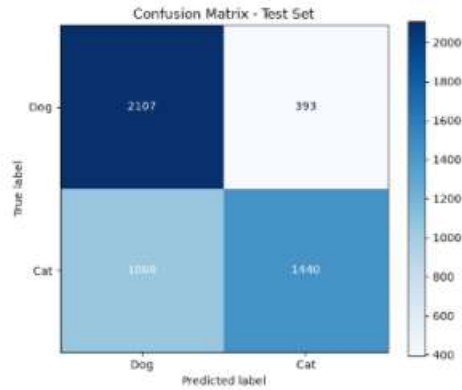
```
[21]: # Train the CNN using only the training portion.
# validation_split=0.20 creates a validation set from X_train/y_train.
history = model.fit(
    X_train,
    y_train,
    batch_size=128,
    epochs=10,
    validation_split=0.2,
    shuffle=True,
    verbose=1
)
```

```
Epoch 1/10
125/125 — 26s 193ms/step - accuracy: 0.5114 - loss: 2.2054 - val_accuracy: 0.5123 - val_loss: 0.6924
Epoch 2/10
125/125 — 26s 204ms/step - accuracy: 0.5255 - loss: 0.6903 - val_accuracy: 0.5225 - val_loss: 0.6911
Epoch 3/10
125/125 — 28s 224ms/step - accuracy: 0.5412 - loss: 0.6859 - val_accuracy: 0.5253 - val_loss: 0.6903
Epoch 4/10
125/125 — 27s 219ms/step - accuracy: 0.5469 - loss: 0.6838 - val_accuracy: 0.5260 - val_loss: 0.6888
Epoch 5/10
125/125 — 29s 235ms/step - accuracy: 0.5530 - loss: 0.6802 - val_accuracy: 0.5617 - val_loss: 0.6786
Epoch 6/10
125/125 — 29s 234ms/step - accuracy: 0.6055 - loss: 0.6584 - val_accuracy: 0.6363 - val_loss: 0.6462
Epoch 7/10
125/125 — 31s 247ms/step - accuracy: 0.6658 - loss: 0.6119 - val_accuracy: 0.6793 - val_loss: 0.6019
Epoch 8/10
125/125 — 29s 234ms/step - accuracy: 0.6988 - loss: 0.5821 - val_accuracy: 0.7005 - val_loss: 0.5746
Epoch 9/10
125/125 — 36s 287ms/step - accuracy: 0.7197 - loss: 0.5539 - val_accuracy: 0.7147 - val_loss: 0.5649
Epoch 10/10
125/125 — 31s 247ms/step - accuracy: 0.7343 - loss: 0.5316 - val_accuracy: 0.7165 - val_loss: 0.5705
```



Confusion Matrix (Test Set):
[[2107 393]
[1060 1440]]

Confusion Matrix (Test Set):
[[2107 393]
[1808 1440]]



Prediction Inference

Now that the model is loaded, we can use it to make predictions on new, unseen data. For demonstration purposes, I'll take a few samples from your existing `X` data to illustrate how to get predictions from the `loaded_model`.

```
[25]: # Select a small subset of the unseen test set for inference
test_samples = X_test[:5]

# Make predictions using the loaded model
inference_predictions = loaded_model.predict(test_samples, verbose=0)

print("Predictions for unseen test samples:")
for i, prediction in enumerate(inference_predictions):
    predicted_class = 'Cat' if prediction[0] > 0.5 else 'Dog'
    probability = prediction[0] * 100
    print(f"Sample {i+1}: Predicted Class = {predicted_class} (Cat probability: {probability:.2f}%)")

# Display the images and predictions
plt.figure(figsize=(10, 4))
for i in range(min(5, len(test_samples))):
    plt.subplot(1, 5, i + 1)
    plt.imshow(test_samples[i].reshape(IMG_SIZE, IMG_SIZE), cmap='gray')
    predicted_class = 'Cat' if inference_predictions[i][0] > 0.5 else 'Dog'
    plt.title(f"Pred: {predicted_class}")
    plt.axis('off')
plt.tight_layout()
plt.show()
```

Predictions for unseen test samples:
Sample 1: Predicted Class = Cat (Cat probability: 84.14%)
Sample 2: Predicted Class = Cat (Cat probability: 85.22%)
Sample 3: Predicted Class = Dog (Cat probability: 12.74%)
Sample 4: Predicted Class = Dog (Cat probability: 46.43%)
Sample 5: Predicted Class = Dog (Cat probability: 33.88%)



Training 3: Localhost Run, Image Size 100, Epoch 10

```
[8]: DATADIR = "/mnt/d/DIT/Bioinformatics/Code/PetImages/PetImages" # <- change this to the path that contains your Dog/ and Cat/ folders
# this is the directory of your datasets
CATEGORIES = ["Dog", "Cat"] # Defining the Label as "Dog" or "Cat"

for category in CATEGORIES:
    path = os.path.join(DATADIR, category) # path to cats or dogs dir
    for img in os.listdir(path):
        img_array = cv2.imread(os.path.join(path, img), cv2.IMREAD_GRAYSCALE)
        if img_array is None:
            # a handful of files in the kaggle dataset are corrupted/unreadable - skip them
            continue

        # reading the image in the path and transforming it into an array
        # cv2.IMREAD_GRAYSCALE converts the image to grayscale so the CNN only has
        # to learn from one color channel instead of three (faster, smaller model)

        plt.imshow(img_array, cmap="gray")
        plt.show()
        break
    break
```

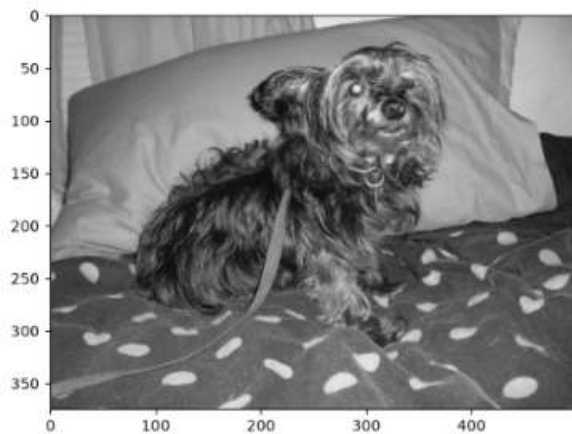


Image Inspection

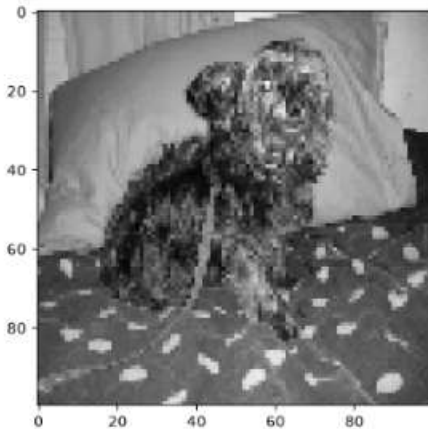
```
[9]: print(img_array)      # Displaying the image in array form (just for checking)
     print(img_array.shape) # checking the resolution of the image

[[117 117 119 ... 133 132 132]
 [118 117 119 ... 135 134 134]
 [119 118 120 ... 137 136 136]
 ...
 [ 79  74  73 ...  80  76  73]
 [ 78  72  69 ...  72  73  74]
 [ 74  71  70 ...  75  73  71]]
(375, 500)
```

Image Resizing

```
[10]: IMG_SIZE = 100 # Initializing variable "IMG_SIZE" as 100

new_array = cv2.resize(img_array, (IMG_SIZE, IMG_SIZE))
plt.imshow(new_array, cmap='gray')
plt.show()
```



Assigning Training Data

```
[11]: training_data = [] # assigning training data

def create_training_data():
    for category in CATEGORIES:
        path = os.path.join(DATA_DIR, category) # path to cats or dogs dir
        class_num = CATEGORIES.index(category)
        for img in os.listdir(path):
            try:
                img_array = cv2.imread(os.path.join(path, img), cv2.IMREAD_GRAYSCALE)
                new_array = cv2.resize(img_array, (IMG_SIZE, IMG_SIZE))
                training_data.append([new_array, class_num])
            except Exception as e:
                pass

create_training_data()
```

```
Corrupt JPEG data: 399 extraneous bytes before marker 0xd9
Corrupt JPEG data: 226 extraneous bytes before marker 0xd9
Corrupt JPEG data: 162 extraneous bytes before marker 0xd9
Warning: unknown JFIF revision number 0.00
Corrupt JPEG data: 1403 extraneous bytes before marker 0xd9
Corrupt JPEG data: 254 extraneous bytes before marker 0xd9
Corrupt JPEG data: 2230 extraneous bytes before marker 0xd9
Corrupt JPEG data: 65 extraneous bytes before marker 0xd9
Corrupt JPEG data: 239 extraneous bytes before marker 0xd9
Corrupt JPEG data: 214 extraneous bytes before marker 0xd9
Corrupt JPEG data: 128 extraneous bytes before marker 0xd9
Corrupt JPEG data: 99 extraneous bytes before marker 0xd9
Corrupt JPEG data: 1153 extraneous bytes before marker 0xd9
```

Checking the Number of Training Data

```
[12]: print(len(training_data))

24997
```

Randomizing/Shuffling the Data

```
[13]: import random
      random.shuffle(training_data)

      for sample in training_data[:10]:
          print(sample[1])
```

```
1
1
0
0
1
1
1
1
1
1
```

```
[15]: pickle_in = open('X.pickle', 'rb')
      X = pickle.load(pickle_in)
```

```
X[1]
```

```
[16]: array([[[[239],
               [239],
               [239],
               ...,
               [224],
               [224],
               [224]],

               [[237],
               [237],
               [237],
               ...,
               [222],
               [222],
               [222]],

               [[238],
               [237],
               [237],
               ...,
               [222],
               [221],
               [222]],

               ...,

               [[115],
               [ 99],
               [ 79],
               ...,
               [216],
               [216],
               [219]],

               [[106],
               [ 87],
               [102],
               ...,
               [211],
               [212],
               [214]],

               [[ 78],
               [ 81],
               [ 70],
               ...,
               [214],
               [217],
               [215]]], shape=(100, 100, 1), dtype=uint8)
```

```
[10]: # Required imports for dataset splitting and evaluation
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, roc_curve, auc

# Ensure labels are a NumPy array
y = np.asarray(y).astype(int)

# Split the dataset into training/development data and a completely unseen test set
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print(f"Total samples:      {len(X):,}")
print(f"Training samples:     {len(X_train):,}")
print(f"Test samples:          {len(X_test):,}")
print("\nThe training data will use validation_split=0.20, so the final proportions are approximately:")
print(" Training:   64%")
print(" Validation: 16%")
print(" Test:        20%")

Total samples:      24,997
Training samples:   19,997
Test samples:       5,000
```

The training data will use validation_split=0.20, so the final proportions are approximately:

```
Training:   64%
Validation: 16%
Test:       20%
```

```
[20]: model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 98, 98, 2)	20
activation (Activation)	(None, 98, 98, 2)	0
max_pooling2d (MaxPooling2D)	(None, 49, 49, 2)	0
conv2d_1 (Conv2D)	(None, 47, 47, 64)	1,216
activation_1 (Activation)	(None, 47, 47, 64)	0
max_pooling2d_1 (MaxPooling2D)	(None, 23, 23, 64)	0
conv2d_2 (Conv2D)	(None, 21, 21, 64)	36,928
activation_2 (Activation)	(None, 21, 21, 64)	0
max_pooling2d_2 (MaxPooling2D)	(None, 10, 10, 64)	0
flatten (Flatten)	(None, 6400)	0
dense (Dense)	(None, 64)	409,664
dense_1 (Dense)	(None, 64)	4,160
dense_2 (Dense)	(None, 1)	65
activation_3 (Activation)	(None, 1)	0

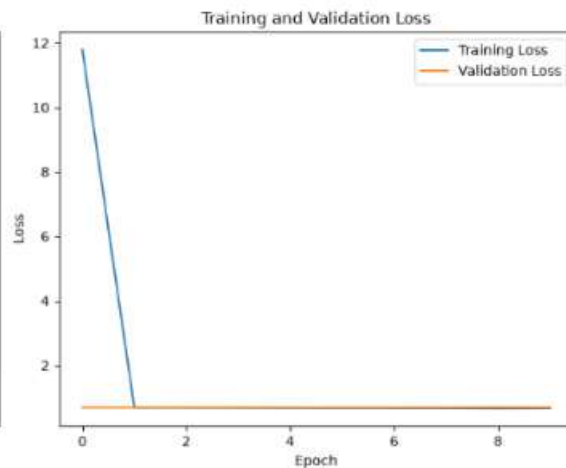
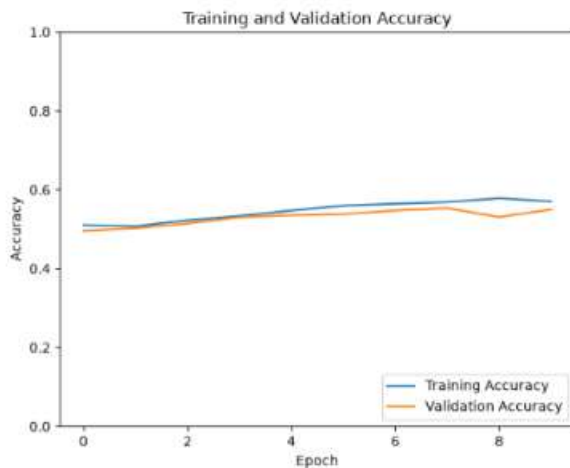
Total params: 452,053 (1.72 MB)

Trainable params: 452,053 (1.72 MB)

Non-trainable params: 0 (0.00 B)

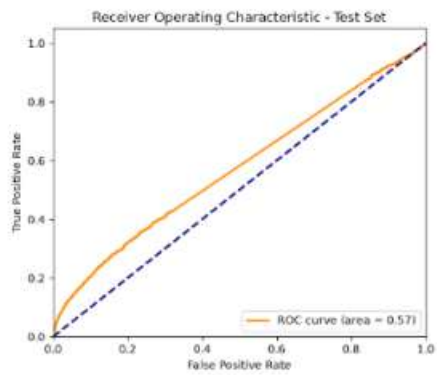
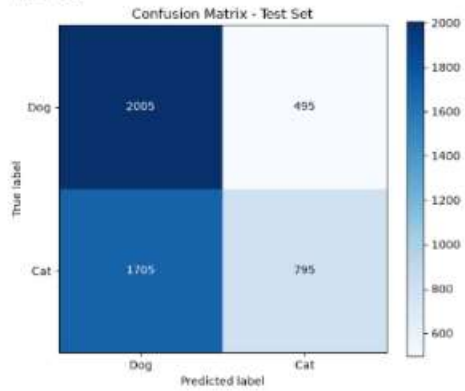
```
[11]: # Train the CNN using only the training portion.
# validation_split=0.20 creates a validation set from X_train/y_train.
history = model.fit(
    X_train,
    y_train,
    batch_size=128,
    epochs=10,
    validation_split=0.2,
    shuffle=True,
    verbose=1
)
```

```
Epoch 1/10
125/125 — 44s 335ms/step - accuracy: 0.5105 - loss: 11.7770 - val_accuracy: 0.4960 - val_loss: 0.6932
Epoch 2/10
125/125 — 56s 445ms/step - accuracy: 0.5077 - loss: 0.6915 - val_accuracy: 0.5025 - val_loss: 0.6931
Epoch 3/10
125/125 — 59s 476ms/step - accuracy: 0.5225 - loss: 0.6893 - val_accuracy: 0.5145 - val_loss: 0.6954
Epoch 4/10
125/125 — 51s 407ms/step - accuracy: 0.5335 - loss: 0.6857 - val_accuracy: 0.5305 - val_loss: 0.6921
Epoch 5/10
125/125 — 55s 441ms/step - accuracy: 0.5474 - loss: 0.6807 - val_accuracy: 0.5355 - val_loss: 0.6888
Epoch 6/10
125/125 — 49s 391ms/step - accuracy: 0.5597 - loss: 0.6753 - val_accuracy: 0.5380 - val_loss: 0.6896
Epoch 7/10
125/125 — 51s 410ms/step - accuracy: 0.5644 - loss: 0.6740 - val_accuracy: 0.5475 - val_loss: 0.6896
Epoch 8/10
125/125 — 50s 401ms/step - accuracy: 0.5606 - loss: 0.6719 - val_accuracy: 0.5535 - val_loss: 0.6934
Epoch 9/10
125/125 — 47s 373ms/step - accuracy: 0.5786 - loss: 0.6664 - val_accuracy: 0.5307 - val_loss: 0.7012
Epoch 10/10
125/125 — 55s 439ms/step - accuracy: 0.5704 - loss: 0.6696 - val_accuracy: 0.5500 - val_loss: 0.6949
```



Confusion Matrix (Test Set):
[[2005 495]
[1705 795]]

Confusion Matrix (Test Set):
[[2005 495]
[1705 795]]



=====

FINAL CNN MODEL EVALUATION

=====

Training Accuracy: 0.5704 (57.04%)
Validation Accuracy: 0.5500 (55.00%)
Test Accuracy: 0.5600 (56.00%)
Test Loss: 0.6857
Precision (macro): 0.5784 (57.84%)
Recall (macro): 0.5600 (56.00%)
F1-score (macro): 0.5326 (53.26%)

=====

Prediction Inference

Now that the model is loaded, we can use it to make predictions on new, unseen data. For demonstration purposes, I'll take a few samples from your existing `X` data to illustrate how to get predictions from the `loaded_model`.

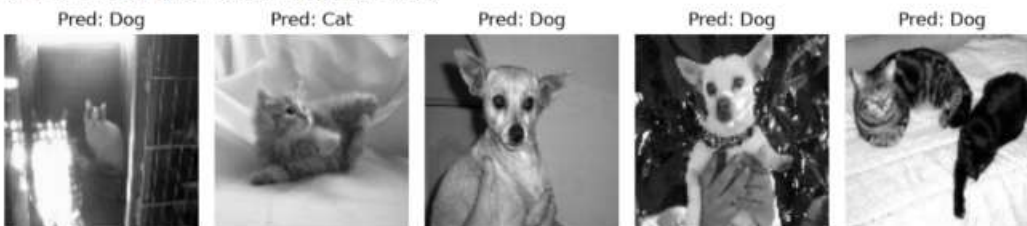
```
[35]: # Select a small subset of the unseen test set for inference
test_samples = X_test[:5]

# Make predictions using the loaded model
inference_predictions = loaded_model.predict(test_samples, verbose=0)

print("Predictions for unseen test samples:")
for i, prediction in enumerate(inference_predictions):
    predicted_class = 'Cat' if prediction[0] > 0.5 else 'Dog'
    probability = prediction[0] * 100
    print(f"Sample {i+1}: Predicted Class = {predicted_class} (Cat probability: {probability:.2f}%)")

# Display the images and predictions
plt.figure(figsize=(10, 4))
for i in range(min(5, len(test_samples))):
    plt.subplot(1, 5, i + 1)
    plt.imshow(test_samples[i].reshape(IMG_SIZE, IMG_SIZE), cmap='gray')
    predicted_class = 'Cat' if inference_predictions[i][0] > 0.5 else 'Dog'
    plt.title(f"Pred: {predicted_class}")
    plt.axis('off')
plt.tight_layout()
plt.show()
```

```
Predictions for unseen test samples:
Sample 1: Predicted Class = Dog (Cat probability: 47.75%)
Sample 2: Predicted Class = Cat (Cat probability: 50.25%)
Sample 3: Predicted Class = Dog (Cat probability: 47.75%)
Sample 4: Predicted Class = Dog (Cat probability: 47.75%)
Sample 5: Predicted Class = Dog (Cat probability: 47.75%)
```



6. Overall Justification / Remarks

Three manual trials were conducted using grayscale input sizes of 50×50 , 80×80 , and 100×100 pixels. The other recorded settings were retained across the trials, including pixel normalization ($X/255.0$), a 3×3 convolution kernel, ReLU and max-pooling layers, a 64-unit dense layer, a one-unit sigmoid output, binary cross-entropy, the Adam optimizer with a learning rate of 0.001, batch size 128, and 10 training epochs. The documented data partition uses a stratified 20% held-out test set and a 20% validation split of the remaining training data (approximately 64% training, 16% validation, and 20% test). This controlled comparison provides a basis for examining the effect of input image size under the recorded training setup.

Among the three reported trials, Trial 2 (80×80 pixels) achieved the highest recorded validation accuracy (0.7165) and test accuracy (0.7063), the lowest recorded test loss (0.5782), and the highest reported macro F1-score (0.7041). Accordingly, the 80×80 configuration is the preferred choice among the three tested image sizes for this experiment. This finding applies to the documented dataset split, architecture, and training settings; it does not establish that 80×80 pixels is universally optimal for dogs-versus-cats classification.

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).